

HERMES AGENT | LESSON 03

How to set up and test advanced Hermes

Profiles, external memory, OpenViking, Hindsight, Codex, subagents, Kanban, Cron, and A2A

Block 1 - Profiles and separate sessions

Profiles are independent Hermes instances within the same installation. Each Profile has its own configuration, sessions, memory, skills, cron jobs, and state.

Remote access and multiple authenticated users will be covered in Lesson 4, together with creating and using Skills and integration with MCP.

Recommendation for this lesson: use the command line to start each Profile. This explicitly gives Hermes the Profile when the process is created and reduces the risk of a session inheriting state, working directory, or context from another Profile.

Desktop and Dashboard remain useful for administration and visualization, but they will not be the main way to open Profile sessions in this lesson. To start a session, prefer:

```
hermes -p <profile> chat
```

Before you begin - if Hermes Desktop appears but does not load on WSL2

On WSL2, Hermes Desktop depends on Linux graphical integration with Windows through WSLg. On some computers, the window may appear on the taskbar or show a warning, but the interface may not load correctly.

First, terminate possible stuck processes in the Ubuntu terminal:

```
pkill -f Hermes  
pkill -f electron  
pkill -f chromium
```

Then close Ubuntu. In Windows PowerShell, run:

```
wsl --shutdown
```

Open Ubuntu again and try to start Desktop:

```
cd ~/hermes-lab  
hermes desktop
```

This procedure restarts the WSL2 graphical environment/processes and is different from the cwd/Profile problem described later in this lesson. If Desktop still does not load, the demonstration can continue through the CLI or Dashboard while the graphical issue is diagnosed.

1.1 Create the `aluno` Profile

1. Open the Hermes Dashboard.
2. Go to ****Profiles****.

3. 3. Select ****Create Profile****.
4. 4. Create a Profile named `aluno`.
5. 5. Add a short description to identify the Profile.

1.2 Configure the `aluno` Profile

If the Profile does not yet have a model/provider configured, run setup:

```
hermes -p aluno setup
```

After setup, start the aluno Profile directly from the terminal:

```
hermes -p aluno chat
```

1.3 Start Profiles from the command line

Whenever you open or switch Profiles during the demonstration, end the current session and explicitly start the desired Profile from the terminal.

```
hermes -p aluno chat  
hermes -p default chat
```

1.4 Confirm separate sessions

6. 6. Start a new `aluno` Profile session with `hermes -p aluno chat`.
7. 7. Send a simple message to identify that session.
8. 8. End the current session and start the main Profile with `hermes -p default chat`.
9. 9. Confirm that the session created in the `aluno` Profile does not appear in the main Profile.
10. 10. Return to the `aluno` Profile with `hermes -p aluno chat` and confirm that the previous session is still available.

1.5 Pay attention to cwd in Hermes Desktop

Hermes Desktop may retain an old working directory (cwd) across sessions or Profile switches. In this situation, changing terminal.cwd in config.yaml may not be enough because Desktop also keeps the remembered value in hermes.desktop.workspace-cwd.

Consequence: a new session may open in the previous Workspace and load context files, skills, memory, or settings associated with the wrong project. For this reason, the command line is the preferred path for the Profile demonstration.

Desktop fix when this problem appears: close Hermes Desktop, confirm the correct terminal.cwd in the Profile's config.yaml, and remove the hermes.desktop.workspace-cwd key from Electron local storage. On Windows, this storage is located at %APPDATA%\Hermes\Local Storage\leveldb\ . Then open Desktop again and create a new session.

This procedure is specific to Desktop. Profile/cwd problems in Dashboard or long-lived gateways have a different origin and should not be fixed by deleting Desktop's LevelDB.

Block 2 - Hermes persistent memory

Hermes built-in memory keeps useful information across sessions and is separated by Profile. It uses MEMORY.md and USER.md internally, but the demonstration can be performed directly through chat.

2.1 Demonstrate memory across sessions

11. 11. In the `aluno` Profile, start a new conversation.
12. 12. Provide a simple preference, for example: "I prefer Python examples whenever code is involved."
13. 13. End the session and open another session in the same Profile.
14. 14. Ask which language should be prioritized in examples.

Block 3 - External memory: concept before installation

Hermes has internal memory and also allows you to connect external memory providers. External memory maintains persistent knowledge beyond built-in memory and can retrieve only the information relevant to each question.

Internal memory remains active alongside the external provider. However, each Profile can have only one active external provider at a time.

3.1 Why external memory can reduce context usage

When large documents are placed directly into the conversation, part or all of their content occupies the model's context window. With on-demand retrieval, the system first searches for relevant information and sends only the necessary excerpt to the model.

This does not increase the model's maximum context window. The benefit is avoiding large volumes of information remaining in context at every interaction, which can reduce context-token consumption.

3.2 OpenViking and Hindsight solve different problems

- **OpenViking:** organizes knowledge in a resource hierarchy and uses layered loading. It is especially useful for retrieving specific parts of documents without loading everything at once.
- **Hindsight:** works with long-term memory, entities, relationships, and synthesis/reflection over stored facts. Its main advantage is structuring and relating knowledge; context savings may occur through on-demand retrieval, but that is not its primary purpose.

Because only one external provider can be active per Profile, the aluno Profile will use OpenViking and the lois Profile will use Hindsight.

Block 4 - OpenViking as external memory

4.1 Install uv on Ubuntu/WSL2

First install uv using Astral's official installer. Run in the Ubuntu/WSL2 terminal:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.bashrc
uv --version
```

If uv --version displays the installed version, continue. If the command is still not found, close and reopen the Ubuntu terminal and test again.

Avoid using `sudo snap install astral-uv --classic` as the primary path. Astral's standalone installer avoids the Snap classic-confinement warning and follows the official uv documentation.

4.2 Install and prepare OpenViking

Choose the installation method according to the embedding type that will be used in OpenViking.

Option 1 - Standard installation

Use this option when embeddings will be configured through a Cloud API or Ollama.

```
uv tool install openviking --upgrade
```

Option 2 - Lightweight CPU embedding

Use this option when you want to select Lightweight CPU embedding in the wizard without depending on Ollama or an external API for embeddings.

```
uv tool install "openviking[local-embed]" --upgrade
```

The local-embed extra adds the dependency required for local llama.cpp-based embeddings.

If OpenViking was already installed with Option 1 and you later decide to use Lightweight CPU embedding, run the Option 2 command to upgrade the installation with that extra.

After installation, start the configuration wizard:

```
openviking-server init
```

In the wizard, choose Step-by-step setup. Under Embedding setup, select Cloud API, Local via Ollama, or Lightweight CPU embedding according to the installation option selected above.

At the VLM step, choose the provider that will be used for interpretation and synthesis. One tested workflow is OpenAI Codex (Subscription), which uses subscription authentication; other options can be used according to your environment.

If Lightweight CPU embedding reports that llama-cpp-python is not installed, do not continue with an incomplete configuration. Exit the wizard, run:

```
uv tool install "openviking[local-embed]" --upgrade
```

Then run `openviking-server init` again.

After completing the wizard, validate the configuration:

```
openviking-server doctor
```

4.3 Use the local server without an API key

When OpenViking runs only on localhost with host 127.0.0.1, an API key is not required. Without `root_api_key` configured, the server automatically enters development mode (dev mode) with authentication disabled.

Minimal local configuration in `~/.openviking/ov.conf`:

```
{
  "server": {
    "host": "127.0.0.1",
```

```
    "port": 1933
  }
}
```

Do not configure `root_api_key` as an empty string. For local use, remove the field. If the server is exposed to the network or another computer, configure authentication with a `root_api_key` before changing the host to a non-local address.

4.4 Start the OpenViking server

```
openviking-server
```

By default, the server uses the configuration in `~/openviking/ov.conf`. During first-time configuration, `init` prepares the file and `doctor` checks whether the local environment is usable.

Keep this terminal open while Hermes is using OpenViking. The server normally starts on port 1933.

4.5 Enable local OpenViking in the `aluno` Profile

Open a second Ubuntu terminal. Keep the `openviking-server` terminal open.

```
hermes -p aluno config set memory.provider openviking
```

Configure the local OpenViking endpoint in the `aluno` Profile `.env` file:

```
echo "OPENVIKING_ENDPOINT=http://127.0.0.1:1933" >> ~/.hermes/profiles/aluno/.env
```

The `hermes -p aluno` memory setup wizard may require an API key even when the local server is in dev mode. For local unauthenticated use, configure the provider and endpoint manually with the commands above.

```
hermes -p aluno memory status
```

Use memory status to confirm that OpenViking is active in the `aluno` Profile.

Important: each Profile can use its own external-memory configuration. In this lesson, the `aluno` Profile will use OpenViking and the `lois` Profile will use Hindsight.

Hermes native persistent memory continues to exist; the external provider adds a specialized storage and retrieval layer.

4.6 Add and query knowledge

15. 15. Start a new ``aluno`` Profile session with ``hermes -p aluno chat``.
16. 16. Add a small resource to OpenViking using the tools exposed by the provider.
17. 17. Ask a specific question about the added content.
18. 18. Ask Hermes to indicate the resource or URI that was queried.

Block 5 - Hindsight and long-term memory

Hindsight will be used in the `lois` Profile because the `aluno` Profile already has OpenViking as its active external provider.

5.1 Prepare the `lois` Profile

If the lois Profile already exists, there is no need to create it again. If it does not exist yet, create the Profile before continuing.

If lois does not yet have a model/provider configured, run:

```
hermes -p lois setup
```

5.2 Install Hindsight Local Embedded manually

To avoid the automatic installer's timeout, preinstall the complete Local Embedded package in Hermes's Python environment:

```
uv pip install --python ~/.hermes/hermes-agent/venv/bin/python hindsight-all
```

The Hermes wizard may attempt to install hindsight-all automatically. In some environments, this installation exceeds the 120-second limit and displays `uv pip install timed out`. The manual installation above avoids depending on that timeout.

5.3 Configure Hindsight in the `lois` Profile

Run:

```
hermes -p lois memory setup
```

In the menu, select Hindsight, then Local Embedded - Run Hindsight locally.

When prompted, choose the LLM provider. The tested workflow uses MiniMax. In Local Embedded mode, embeddings and reranking run locally.

If the wizard still tries to install hindsight-all and times out, complete or cancel that step, run the manual installation command again, and repeat `hermes -p lois memory setup`.

5.4 Older GPUs or CUDA incompatibility: force CPU

If retain or recall fails with `cudaErrorNoKernelImageForAvailable`, the local embedding/reranking backend attempted to use a CUDA GPU incompatible with the available kernels.

First configure Hindsight's embedded runtime for CPU:

```
hindsight-embed profile create hermes --merge --env HINDSIGHT_API_EMBEDDINGS_LOCAL_FORCE_CPU=true  
--env HINDSIGHT_API_RERANKER_LOCAL_FORCE_CPU=true
```

If the same variables also need to be registered in the lois Profile, run:

```
echo "HINDSIGHT_API_EMBEDDINGS_LOCAL_FORCE_CPU=true" >> ~/.hermes/profiles/lois/.env  
echo "HINDSIGHT_API_RERANKER_LOCAL_FORCE_CPU=true" >> ~/.hermes/profiles/lois/.env
```

Check:

```
tail -n 10 ~/.hermes/profiles/lois/.env
```

Then close any running Hermes/Hindsight sessions and reopen:

```
hermes -p lois chat
```

5.5 Confirm Hindsight is active

```
hermes -p lois memory status
```

The status should indicate Hindsight as the active external provider in the lois Profile.

5.6 Test retain, recall, and reflect

Open the lois Profile:

```
hermes -p lois chat
```

In chat, explicitly ask Hermes to use `hindsight_retain` and store a test fact. Then end the session, open a new one, and ask it to use `hindsight_recall` to retrieve the information.

To test synthesis across memories, store two related pieces of information and ask it to use `hindsight_reflect` to summarize the relationships found.

Note: retain may be asynchronous; wait a few seconds before the first recall when the memory has just been written.

5.7 Create relationships and retrieve knowledge

Record related facts, for example: Ana coordinates the Hermes Course; Bruno maintains an API used by the course; the course uses OpenViking for documentation.

Ask the agent to retrieve the relationships among the registered entities.

Ask for a reflection that separates recorded facts from inferences.

Block 6 - Use Codex in the same `aluno` Profile

You do not need to create a dedicated Profile for Codex. The Codex runtime can be enabled in the same aluno Profile used with OpenViking. The Profile remains the same; only the runtime used for that type of session changes.

In the `codex_app_server` runtime, tools that depend directly on Hermes's internal loop, such as `delegate_task` and the memory tool, are unavailable during the turn. After the Codex demonstration, return to the default runtime before continuing with memory or subagents.

6.1 Install and authenticate Codex CLI

```
npm i -g @openai/codex
codex --version
codex login
```

Use Codex CLI 0.130.0 or later.

6.2 Prepare a Workspace for the test

```
mkdir -p ~/hermes-lab/aula3-codex
cd ~/hermes-lab/aula3-codex
pwd
```

19. Stay in the directory returned by ``pwd``.

20. Start the `aluno` Profile from this Workspace in the terminal.

21. Run ``hermes -p aluno chat``.

6.3 Enable the Codex runtime

In the aluno Profile chat:

```
/codex-runtime  
/codex-runtime codex_app_server
```

Open a new session after enabling it.

6.4 Run a coding task

22. 22. Ask Hermes to confirm the current directory and list the Workspace files.
23. 23. Ask it to create a small Python project with a main file, tests, and README.
24. 24. Follow the changes directly in the Workspace directory through the terminal or file editor.
25. 25. Run the tests from the terminal only if necessary.

6.5 Return to the default runtime

```
/codex-runtime auto
```

Open a new session before the next blocks.

Block 7 - Subagents

Subagents are temporary executions created by `delegate_task`. They receive a task and an isolated context, perform the work, and return the result to the main agent. They are not persistent Profiles.

7.1 Demonstrate a subagent

26. 26. In the terminal, use a session for the desired Profile started with ``hermes -p <profile> chat`` and keep the default runtime.
27. 27. Ask the agent: "Delegate to a subagent the task of comparing persistent memory and conversation context. Return only the final summary."
28. 28. Follow the execution through the agents interface when available.

Block 8 - Kanban: create and track a project

Kanban organizes durable tasks on a board and records work progress. In this demonstration, all steps will be performed by the previously created aluno Profile.

8.1 Open Kanban

29. 29. Open `**Dashboard > Kanban**`.
30. 30. If the database does not yet exist, simply opening Kanban may initialize it automatically. If manual initialization is required:

```
hermes kanban init
```

8.2 Create the project in Kanban

Project: research a Hermes technical update, produce an educational summary, and review the result before completion.

31. 31. Keep `**Orchestration**` set to `**Manual**` for the first test.
32. 32. Create a main task in `**Triage**` to represent the project.
33. 33. Create the work stages: research the topic, produce the material, and review the result.
34. 34. Define dependencies among the stages.

35. 35. Assign all tasks to the `aluno` Profile.
36. 36. Check the goal, Workspace, and acceptance criteria for each task.
37. 37. Move the first task to **Ready**.
38. 38. Use **Nudge dispatcher** to run a cycle immediately when necessary.
39. 39. Track the task through **Ready**, **Running**, and **Done** or **Blocked**.
40. 40. When a stage finishes, follow the release of the next dependent task.
41. 41. Repeat the flow until the project is complete.
42. 42. Open the cards to view history, execution, comments, and result.

8.3 Test automatic orchestration

43. 43. After confirming the manual flow, create another main task in **Triage**.
44. 44. Use **Decompose** to generate subtasks.
45. 45. Check whether the subtasks and dependencies make sense and whether the `aluno` Profile remained responsible.
46. 46. Only after this check, test **Orchestration: Auto**.

Block 9 - Cron and automations

Cron allows you to create one-time or recurring tasks. They can be created in natural language by the agent itself and monitored through Desktop or Dashboard.

9.1 Create an automation

47. 47. Open a terminal conversation with the chosen Profile, for example: `hermes -p aluno chat`.
48. 48. Ask: "In 5 minutes, create a short summary of the current state of this project and deliver it locally."
49. 49. Open the **Cron** area in Hermes Desktop or Dashboard.
50. 50. Confirm name, schedule, state, and destination.
51. 51. Use **Trigger now** to test immediately.
52. 52. Test **Pause**, **Resume**, **Edit**, and **Delete** through the interface.

Cron jobs run in new sessions. The prompt must contain the context necessary to execute the task independently.

Block 10 - A2A: communication between independent agents

A2A is a communication protocol between independent agents. One Hermes can call another Hermes as an agent, discover its capabilities, and maintain an A2A conversation through context_id.

A2A is not required for Kanban. For multiple Profiles on the same machine, Kanban already provides a durable work queue. A2A is more appropriate when communication must cross independent processes, machines, or frameworks.

10.1 Example: main Profile calling the `lois` Profile

The example uses the main Profile as the A2A client and the lois Profile as the called agent. The same flow can be reproduced with other Profile names.

10.2 Enable A2A in the `lois` Profile

Run the lois Profile gateway setup and enable A2A:

```
hermes -p lois gateway setup
```

Define a dedicated local port for lois A2A, for example 9901.

Start the lois gateway explicitly from the terminal:

```
hermes -p lois gateway run
```

10.3 Enable A2A tools in the main Profile

53. 53. In the main Profile, open the ****Tools/Toolsets**** configuration.

54. 54. Enable the ``a2a`` toolset.

55. 55. Register the ``lois`` Profile as a known agent pointing to the local A2A endpoint.

10.4 Call the ``lois`` agent

56. 56. In the main Profile chat started from the command line, ask it to discover the capabilities of the ``lois`` agent.

57. 57. Send a short task to ``lois``.

58. 58. Ask a second question while keeping the same ``context_id`` to demonstrate continuity in the agent-to-agent conversation.

59. 59. Consult the A2A conversation history when necessary.

Block 11 - Clean up providers and delete Profiles

11.1 Disable Hindsight in the ``lois`` Profile

Before removing local files, return the lois Profile to built-in memory:

```
hermes -p lois memory off
```

Expected output: Memory provider: built-in only.

11.2 Remove local Hindsight components

Remove the client installed by setup, when present:

```
uv pip uninstall --python ~/.hermes/hermes-agent/venv/bin/python hindsight-client
```

If hindsight-all is installed in the same environment, remove it too:

```
uv pip uninstall --python ~/.hermes/hermes-agent/venv/bin/python hindsight-all
```

If the command reports that hindsight-all is not installed, continue normally.

Delete only the embedded PostgreSQL instance created by Hindsight:

```
rm -rf ~/.pg0/instances/hindsight-embed-hermes
```

Remove `HINDSIGHT_*` variables from the lois Profile:

```
sed -i '/^HINDSIGHT_/d' ~/.hermes/profiles/lois/.env
```

Check that no variables remain:

```
grep HINDSIGHT ~/.hermes/profiles/lois/.env
```

11.3 Remove OpenViking installed by uv

```
uv tool uninstall openviking
```

Confirm that it no longer appears in the list:

```
uv tool list
```

To also delete local OpenViking configuration and data:

```
rm -rf ~/.openviking
```

To delete the local cache/model downloaded by OpenViking when you want to fully reset the environment:

```
rm -rf ~/.cache/openviking
```

11.4 Delete Profiles

The command below permanently removes the specified Profile, including its configuration, memory, sessions, and skills. The default Profile must not be removed.

```
hermes profile delete <profile-name> --yes
```

Examples used in this lesson:

```
hermes profile delete aluno --yes  
hermes profile delete lois --yes
```

Confirm through the CLI:

```
hermes profile list
```

If aluno or lois no longer appears in the list, deletion is complete. If Dashboard still shows a stopped entry, reload or restart Dashboard to refresh the interface.

Lesson 4

Remote access and multiple users, creating and using Skills, and integration with MCP servers.

Official references

- Profiles: <https://hermes-agent.nousresearch.com/docs/user-guide/profiles/>
- Profile Commands: <https://hermes-agent.nousresearch.com/docs/reference/profile-commands/>
- Hermes Desktop cwd issue #38855: <https://github.com/NousResearch/hermes-agent/issues/38855>
- Hermes Desktop profile/cwd issue #79406: <https://github.com/NousResearch/hermes-agent/issues/79406>
- Local Hermes Profiles issue #81492: <https://github.com/NousResearch/hermes-agent/issues/81492>
- Memory: <https://hermes-agent.nousresearch.com/docs/user-guide/features/memory/>
- Memory Providers: <https://hermes-agent.nousresearch.com/docs/user-guide/features/memory-providers/>
- OpenViking Quick Start - Server Mode: <https://github.com/volcengine/OpenViking/blob/main/docs/en/getting-started/03-quickstart-server.md>

- Codex App-Server Runtime:
<https://hermes-agent.nousresearch.com/docs/user-guide/features/codex-app-server-runtime>
- Subagent Delegation:
<https://hermes-agent.nousresearch.com/docs/user-guide/features/delegation/>
- Kanban: <https://hermes-agent.nousresearch.com/docs/user-guide/features/kanban>
- Kanban Tutorial: <https://hermes-agent.nousresearch.com/docs/user-guide/features/kanban-tutorial>
- Cron: <https://hermes-agent.nousresearch.com/docs/user-guide/features/cron/>
- A2A: <https://hermes-agent.nousresearch.com/docs/user-guide/messaging/a2a>
- Hindsight plugin: <https://github.com/NousResearch/hermes-agent/blob/main/plugins/memory/hindsight/README.md>
- Hindsight FORCE_CPU example: <https://github.com/NousResearch/hermes-agent/issues/8972>
- Hindsight environment variables:
<https://github.com/vectorize-io/hindsight/blob/main/.env.example>
- OpenViking Quick Start: <https://github.com/volcengine/OpenViking/blob/main/docs/en/getting-started/03-quickstart-server.md>

Content translated from Portuguese into English by AI.